

從聽不懂， 到能交代清楚

先替一堆縮寫搭一座鷹架，再看 AI 怎麼真的動手做事，最後決定用哪一種容器交出去。



CHANG JUI-MING

—— 張睿明 / AIMing Data



本堂路線

01

替名詞搭鷹架

用一次大掃除，替 Agent、MCP、RAG 找到位置。

02

從會說到會做

模型如何提出工具請求，再由應用程式執行。

03

選對容器

Markdown、JSON、HTML 各自適合交給誰；YAML 在技能檔中帶過。

同一個案例，從交代任務到驗收成果。 | 先定位 → 再看執行 → 最後選交付方式



AI News this week | 這禮拜的三則

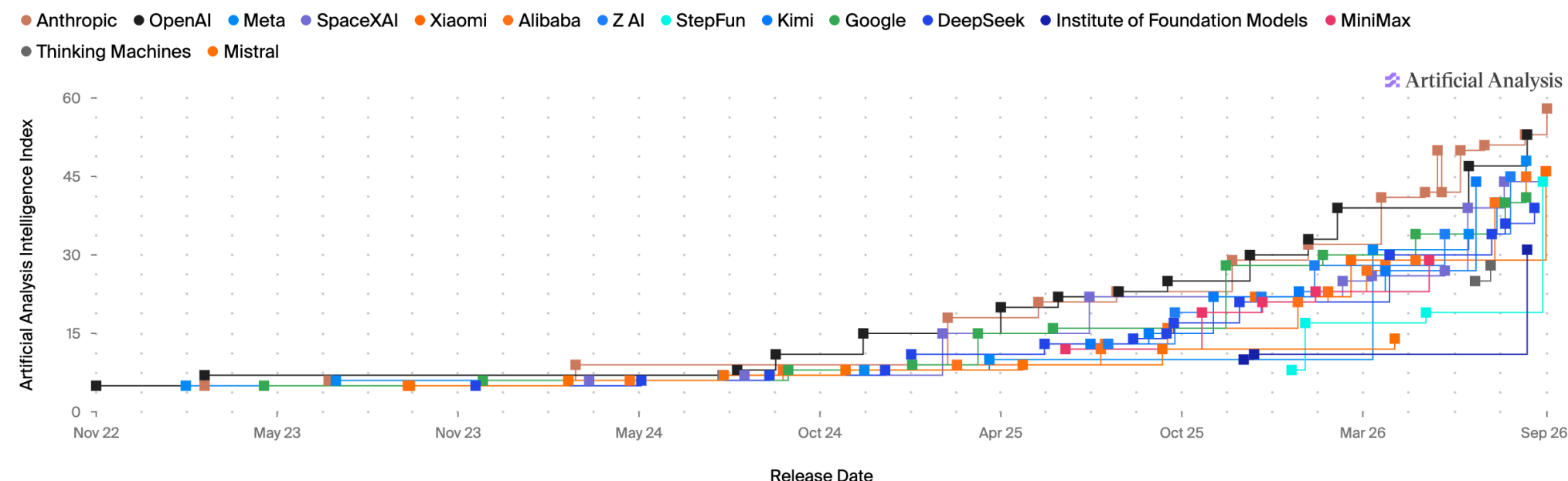
開場暖身：先看看這禮拜 AI 圈冒出來的東西。



Claude Opus 5.5性能升級、成本降40% OpenAI 火速推出GPT-6 Sol、Luna

Frontier Language Model Intelligence, Over Time

Artificial Analysis Intelligence Index v4.3.2 incorporates 10 evaluations: AA-Briefcase v1.1, GDPval-AA v2.1, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1



Thariq
@trq212



翻譯自英文 顯示原文

這裡有很多回饋，你們許多人正在自己規劃 & 不需要計劃模式

其他人則偏好進入一個模式的 UX，在那個模式中 Claude 只是跟你一起思考 & 腦力激盪

我的計劃是：

- 將計劃模式變成一個內建模組
- 允許模組新增模式或覆寫 shift+tab

評價此翻譯：



今天用這一件事，串起三個主題

課堂虛構案例

整理三份會議紀錄，列出誰要做什麼，做成一頁報告，再草擬提醒信，

但**不要寄出**。

共同輸入 | 三份會議紀錄

A | 阿明整理雨量資料，期限為 2026 年 10 月 8 日。

B | 怡君製作調查簡報，期限尚未決定。

C | 兩項工作的完成狀態，都還需要向負責人確認。

三份紀錄共同用於以下三個主題，不是 A、B、C 各對應一個主題。

名詞鷹架

「把三份紀錄作為 Context」
名詞有助溝通，卻不容易學；
用類比搭起理解的鷹架。

Function Calling

若紀錄在雲端，調用已授權的
雲端讀檔工具取得內容；
只草擬提醒信，不調用寄信工具。

格式

輸出報告、結構化資料或信稿；
保留未定期限與待確認狀態。



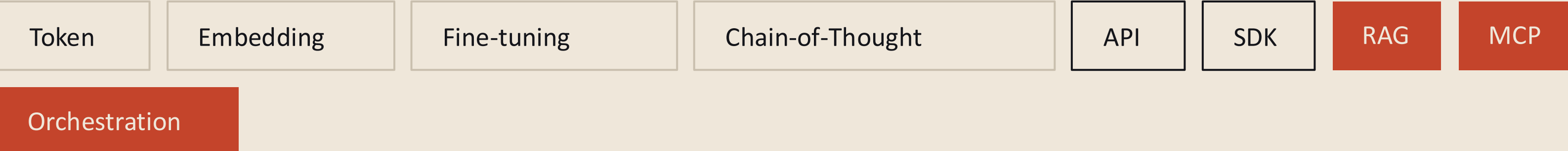
01

替名詞搭一座鷹架

不先背定義，先替整個領域找一個暫時可用的
框架。



每個字單獨看都認識， 放在同一句話裡卻完全看不懂。



你查一個不懂的詞，解釋裡又冒出三個新詞。原本只是想知道 Agent 是什麼，最後卻像誤闖一場縮寫聯誼會——而且每個人都只肯用英文名字自我介紹。

| 不是逐字翻譯 → 是還沒看到彼此的關係



難的不是翻譯， 是這個詞沒有位置

MCP · MODEL CONTEXT PROTOCOL

一種讓 AI 應用以標準方式連接外部工具、資料來源與服務的協議。

讀完之後腦中只剩下：好，這是一種.....協議。

像第一次走進大型五金行：每件東西都有名字，但你不知道哪一區是水電、哪一區是木工，也不知道手上面的問題需要螺絲起子還是老虎鉗。

它屬於哪一區？

模型、推理、工具、記憶、評估，還是安全？

先找區域 → 再看它在流程裡做什麼

它在流程裡做什麼？

先有位置，再補精確定義。



類比不是答案，而是一座臨時鷹架

Token
樂高積木

Token ID
置物櫃號碼

Chain-of-Thought
計算紙

這些說法都不完整，但能先讓腦中出現一個可操作的畫面。等理解加深，再把「MCP 像萬用接頭」換回正式定義。

類比不是要取代定義，而是讓我們有機會走到定義面前，而不是在半路就關掉網頁。

計算紙只是類比；看得見的解釋，不等於模型內部計算的完整紀錄。

先有畫面 → 再回定義 → 記得類比的限制



一次大掃除，三種工作方式

既定流程 流程主要事先決定

按照事先設計的步驟與條件分支執行。

你逐步指揮 人持續主導下一步

你看結果，再交代接下來做什麼。

在限制內決定下一步 依結果調整

Agent根據現場與回傳結果，選擇行動、檢查或請示。

「明天下午有人來家裡，請把客廳整理到可以招待客人的程度。」

Agent 可以只有一個；多 Agent 是另一種分工設計。

比較誰決定下一步，不比較有幾個人



把名詞放回大掃除的現場：誰在做事

名詞	大掃除類比	在系統中的角色
LLM	受過訓練的大腦	理解指令、生成文字、判斷下一步
Agent	被分派任務的清潔專家	根據目標採取行動並回報結果
Manager Agent	清潔領班	拆解任務、分派工作、整合結果
Goal	整理到能招待客人	系統最後要達成的結果
Planning	先擦桌面還是先吸地	安排完成目標所需的步驟
Task Decomposition	拆成丟垃圾、擦窗、拖地	把大任務切成可執行的小任務
Workflow	一張大掃除流程表	定義順序、判斷與交接

Manager Agent：需要協調分工時可採用，不是每個 Agent 系統都必備。

目標是什麼 → 步驟怎麼拆 → 是否需要分工



拿什麼做、記什麼、誰把關

名詞	大掃除類比	在系統中的角色
Tool	掃把、吸塵器、梯子	讓 AI 能操作外部功能
Function Calling	領班填工單叫師傅	用結構化資料要求程式執行功能
MCP	可連不同設備的標準接頭	用一致方式連接工具與資料
RAG	翻閱清潔手冊與維修紀錄	先找外部資料，再據以回答
Context	攤在工作桌上的資訊	模型這一輪可以參考的內容
Memory	工作紀錄與屋主偏好	保存之後可能再用到的資訊
Guardrails	禁區與安全守則	限制系統不能做什麼
Human-in-the-loop	碰到保險箱請屋主決定	關鍵節點交回人類確認

| 功能 → 可參考的資訊 → 行為邊界



把大掃除的名詞，搬回一份真實工作

名詞	在這個案例裡是什麼
Goal 目標	整理出工作分工、一頁報告與提醒信草稿。
Agent 推進任務的系統	依結果決定下一步，必要時停下來。
Skill 做事的方法	整理會議紀錄的步驟、規則與可用資源。
Tool 可呼叫的功能	讀取文件、儲存報告。
Context 這輪可參考的資料	任務要求與已提供、已讀取的文件內容。
Guardrails 行為邊界	只讀指定資料、不補寫未知、不寄信。

工具回答「能做什麼」；技能整理「這類事情怎麼做」。

這是教學上的角色分工，不是所有系統必備的六個模組。

掃把變成讀檔功能 → 清潔方法變成整理方法 → 名詞的位置不變



一句組合技，先翻回生活語言

Manager Agent 會根據 Goal 進行 Task Decomposition，再透過 MCP 呼叫 Tools，使用 RAG 取得資料，最後由 Human-in-the-loop 進行確認。



領班先理解屋主要什麼，把大掃除拆成幾項工作，再透過標準接頭叫用需要的設備，翻閱手冊與過去的清潔紀錄，最後把高風險的決定交回屋主確認。

看懂之後再換回正式名詞——同一句話就從密碼變成一個可以想像的工作流程。

其中一種系統搭配，不代表每個任務都需要 MCP、RAG 或 Manager Agent。

讀一次名詞版 → 對照生活版 → 看角色如何合作



不必背完：需要時回到名詞圖鑑

01
基礎設施與開發層

模型、Agent、API、框架與系統編排

02
思考、推理與規劃

目標、拆解、推理、反思與自主性

03
工具、技能與資源

Tool Use、RAG、知識庫與外部連接

04
記憶與上下文

Context Window、短期／長期記憶與狀態

05
效能、評估與異常

成本、延遲、評估、幻覺與可觀測性

06
安全、防護與治理

權限、護欄、提示注入、隱私與稽核

卡片順序：一句話說明 → 大掃除類比 → 正式定義 → 相關名詞。 | 先找區域 → 再讀一張卡 → 沿著相關詞往下看

名詞圖鑑結構示意 | geomingical.github.io/Agentic_AI_Glossary/



遇到陌生詞，跑這個循環

01 先不要急著背完整定義

03 讀生活類比，先讓腦中出現畫面

05 連接相關名詞，看它和前後怎麼合作

02 找到它的位置：屬於哪一區？

04 回到正式定義，確認類比省略了什麼

06 實際用過之後，再回來看一次

同一個名詞在不同階段看起來會不太一樣。這不是原本沒學會，而是理解正在增加解析度。

| 找到位置 → 確認類比 → 接回正式定義



類比也會騙人， 要知道它從哪裡失效

Agent 像清潔專家

但它沒有人的生活經驗與責任感

RAG 像查手冊

但檢索可能找錯文件， 模型也可能讀錯

CoT 像計算紙

但可見的文字不一定反映內部計算

每個類比後面都要接著問：它和真正的技術概念， 哪裡相似？又從哪裡開始不一樣？

每個類比都要知道， 從哪裡開始不像

Memory 像記憶

但它可能只是資料庫裡的一筆紀錄

MCP 像萬用接頭

它是協議， 不會自動帶來資料權限， 也不保證工具或答案正確。



先接成一張網，不必一次背完

專家看到的

RAG、MCP、Agent、Context、Guardrails 是一張彼此連接的系統圖。

初學者看到的

五個需要單獨背誦的英文縮寫。

先問它屬於哪一區、在流程裡做什麼，再補完整定義。

位置找到了，接著看這些角色怎麼真的合作。

TAKEAWAY | 先有關係，再增加定義的解析度



02

從會說到會做

Function Calling：領班填好工單，請真正會操作工具的人來處理。



不是每個問題都需要工具

Context 裡已經有

改寫一段文章

摘要已貼進對話的內容

解釋已有足夠依據的概念

把資料先整理成表格文字

需要外部資料或行動

查現在的天氣或最新新聞

讀取行事曆與雲端檔案

查公司資料庫裡的訂單

寄出 Email、建立會議或寫入檔案

模型可以生成一句「信件已寄出」，但句子寫得再像物流通知，也不代表信真的離開寄件匣。

資訊是否已提供 → 是否需要外部資料 → 是否真的要改變外部狀態



AI 怎麼知道有哪些工具？先看工具說明書

本次可用的工具

工具	用途	必須提供
read_document	讀取指定文件	document_id：文件識別碼
save_report	儲存整理後的報告	filename：檔名 content：內容

本任務不開放

send_email

可能存在寄信功能，
不代表這次有權使用。

Schema | 工單規格

要填 document_id，值必須是文字。

Arguments | 本次填入內容

document_id = "meeting_A"

知道工具用途、填對欄位，還要有使用權限。

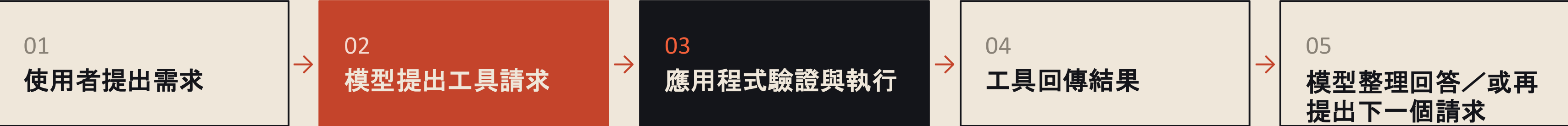
簡化工具規格示意，非特定平台 API。

有哪些功能 → 工單要填什麼 → 本次准不准用

GPT增益集 Demo



它不是讓模型突然長出雙手



模型只是提出工具名稱和參數；真正執行操作的，通常是外部應用程式。

誰開單 → 誰執行 → 誰把結果整理給你



一張工單長什麼樣

「今天台北適合帶傘嗎？」



```
{  
  "name": "get_weather",  
  "arguments": { "location": "Taipei" }  
}
```

工具回傳

地點：台北
降雨機率：80%

模型整理：降雨機率偏高，建議帶傘。
課堂模擬資料，非即時天氣預報。

不同平台可能稱 Function Calling、Tool Calling 或 Tool Use；本堂
著重共同的合作流程。

工單 → 資料 → 建議



一次成功的工具使用，拆成三份東西

1. 模型提出工單

```
{
  "name": "read_document",
  "arguments": {
    "document_id": "meeting_A"
  }
}
```

應用程式執行



2. 執行後，工具回傳資料

已成功讀取：會議紀錄 A。

阿明負責整理雨量資料，期限是 2026 年 10 月 8 日。

根據回傳內容整理



3. 模型整理回答

阿明

工作：整理雨量資料
期限：2026-10-08
來源：會議紀錄 A

請求不是資料；回條不是最後答案；最後答案還要核對。

教學流程模擬，非實際工具執行紀錄。

先看請求 → 再看回條 → 逐欄核對回答



同一張工單，不一定拿到同一種回條

共用請求：讀取 meeting_A

工具回條	合理的後續處理	不應該發生的事
成功讀到文件	根據內容整理，保留來源。	把原文沒有的事補進去。
找不到指定文件	說明找不到，確認位置或識別碼。	自己編一份「可能的紀錄」。
沒有存取權限	說明限制，在授權邊界停下。	假裝已讀取，或繞過權限。

查不到，不等於不存在；不能讀，不等於已經讀完。

工具成功 ≠ 摘要一定正確；工具失敗，也不能接著編答案。

三種課堂模擬情境，不是產品測試結果。 | 只換回條 → 跟著改下一步 → 不把限制寫成答案



工具能不能用， 由三層決定

第一層	產品	決定工具箱裡有什麼、能存取什麼、誰有權限。在本次產品與授權範圍內，不能只靠提示詞跨越。
第二層	系統設定	開發者決定模型可以自行判斷、必須使用、禁止使用，或只能用指定工具。
第三層	你的指令	在可用範圍內影響這一輪的判斷：資料從哪裡來、做到哪一步、哪裡要停。

自然語言是方向盤，不是萬能通行證。 | 有沒有功能 → 本次開放哪些 → 指令要求做到哪裡



四句可以直接用的指令

「請搜尋今天的官方公告，再回答我。」

指定資料來源

「只根據我上傳的檔案回答，不要搜尋網路。」

限制範圍

「先幫我草擬信件，不要寄出。」

停在哪一步

「查完行事曆後列出選項，等我確認再建立會議。」

保留人類確認

你不需要寫 Function，但可以明確講出資料來源、範圍、行動邊界與人類確認。

導讀：來源 → 範圍 → 行動邊界 → 人類確認



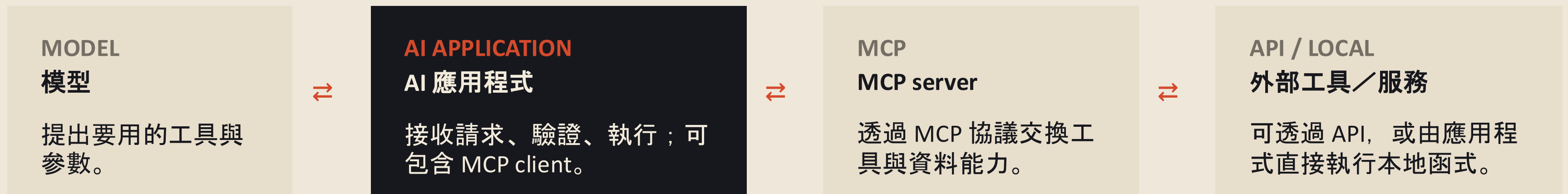
一般使用者要看的四件事

來源	權限	確認	結果
它用了什麼資料？有沒有引用？	要求的授權範圍合理嗎？	高風險動作有沒有先問過你？	能不能在外部系統裡驗證？

實際工具紀錄與外部結果，比模型事後自述更能用來核對；仍須檢查是否符合原需求。
導讀：看依據 → 看權限 → 看確認 → 看實際結果



Function Calling、MCP、API 各站在哪裡？



Function Calling：模型提出工具與參數，由應用程式處理。

MCP：AI 應用與 MCP server 交換能力的標準協議。

API：軟體開放給其他程式使用的操作介面。

這是其中一種接法，不代表每個工具都必須經過 MCP。
也可以由應用程式直接執行本地函式，或直接呼叫外部 API。

模型開工單 → 應用程式接手 → 依接法使用外部能力



會使用工具，也代表有機會用錯工具

講錯天氣，我們白帶一把傘；但如果能寄信、付款、刪檔案或改資料庫，錯誤參數就不只是小尷尬。

假設情境

「幫我刪掉昨天建立的測試檔案。」

↓ 「你確定是這 347 個檔案嗎？」

執行前檢查

驗證工具參數

檢查使用者權限

限制工具可存取的資料

對高風險操作再次確認

執行後追蹤／異常處理

保存操作紀錄 | 處理逾時、失敗與異常回傳

先確認准不准做 → 再驗證做對沒有



從一次呼叫，到一個會繼續判斷的 Agent

工作進度

- 01 讀 A：取得阿明的工作與期限。
- 02 讀 B：取得怡君的工作，期限未定。
- 03 讀 C：兩項工作的完成狀態都待確認。
- 04 整理報告：保留未知，不宣稱工作完成。
- 05 提醒文字草稿：到此停止，不寄出。

停止也是正確的決定；自主不等於無限制。

讀到什麼 → 下一步怎麼變 → 什麼時候該停

AGENT LOOP

觀察結果 → 決定下一步

Observation



再觀察 ← 使用工具

只是整理摘要

期限未知，可標示待確認。

接著建立指定時間活動

缺少必要日期，應先釐清，不能硬填。



不是看它說得多像，而是看事情做到哪一步

請求

模型提出了什麼工單？

確認工具名稱、參數與要求的行動。

執行

工具有沒有執行、回傳什麼？

找真正的回條與外部紀錄。

驗收

結果是否符合來源、需求與權限？

流暢文字不能取代核對。

自然語言與外部系統建立合作方式；但權限、驗證與停止條件仍要有人設計。

TAKEAWAY | 做事的流程清楚了，接著決定成果怎麼交出去



03

選一個不容易誤會的容器

重點不是哪個格式看起來最專業，而是下一位接手的人是誰。



為什麼工單長得像這樣

「幫我安排 2026 年 10 月 1 日，台灣時間下午兩點，和小明開一小時的會。」



```
{  
  "title": "與小明開會",  
  "start_time": "2026-10-01T14:00:00+08:00",  
  "duration_minutes": 60  
}
```

程式通常不想靠氣氛猜答案——它需要清楚的欄位、數值與邊界。
欄位結構示意，不代表已建立行事曆活動。日期與時區要有依據。

哪一天 → 哪個時區 → 多久；先把欄位說清楚



JSON 寫出來了，不代表事情做完了

看到什麼	代表什麼	還不能證明什麼
包含主旨、內文的 JSON	信件資料整理好了。	沒有證明使用過寄信工具。
要求呼叫寄信工具的訊息	系統被要求採取行動。	沒有證明請求已成功執行。
工具執行結果與外部紀錄	可以進一步驗證執行狀況。	仍要核對對象、內容與需求。

我們這次只要草稿，因此不應進入寄信步驟。

Structured Output／結構化輸出：依約定結構整理回覆。

資料 → 請求 → 執行證據；本例停在草稿



這頁看輸入：把交代分清楚。

只選一種先學，我會選 Markdown

幫我整理資料，不要增加內容，使用繁體中文，每一列都要有來源，沒有資料就寫未知，最後用表格輸出。

資訊沒有增加，但「要做什麼、不能做什麼、長什麼樣子」被分開了。格式不保證模型照做，卻能減少它要猜的地方。

改成有分區的寫法

任務

整理提供的資料。

限制

- 不得增加原文沒有的內容
- 使用繁體中文
- 沒有資料時填寫「未知」
- 每一列保留來源

輸出格式

任務、限制、輸出分開寫，減少對方要猜的地方



同一份資料，交給人繼續修改

Markdown 原文

工作整理

已確認的分工

- 阿明：整理雨量資料，期限 2026-10-08。來源：A。
- 怡君：製作調查簡報，期限待確認。來源：B。

尚待確認

- 兩項工作的完成狀態。來源：C。

閱讀時的樣子

工作整理

已確認的分工

- 阿明：整理雨量資料，期限 2026-10-08。來源：A。
- 怡君：製作調查簡報，期限待確認。來源：B。

尚待確認

- 兩項工作的完成狀態。來源：C。

A／B／C＝會議紀錄 A／B／C

現在的目標不是做網站，而是讓下一次修改很直接。

同一份文字 → 分清主題 → 留給下一次修改



JSON：不一定要手寫，但最好看得懂

```
{ }
```

包住一個物件

```
[ ]
```

表示一串項目

```
"city": "Taipei"
```

欄位名稱與它的值

如果 AI 把城市填成 Tokyo，或把「先草擬、不要寄出」轉成 send_email，格式再完美也做錯事情。看得懂欄位和值，至少能在工單送出去前發現它寄錯部門。

結構正確不代表內容正確。格式解決的是邊界問題，不會自動消滅幻覺。

看欄位 → 看值 → 回到來源核對



欄位都填好了，內容仍可能是錯的

同一份資料，交給程式繼續處理

會議紀錄 B：怡君負責製作調查簡報，期限尚未決定。

保留未知的版本

```
{
  "owner": "怡君",
  "task": "製作調查簡報",
  "due_date": null,
  "completion_status": "unknown",
  "sources": ["會議紀錄 B", "會議紀錄 C"]
}
```

能不能解析 → 欄位是否符合約定 → 內容是否有來源依據

本例約定：null 表示期限未確認；不是漏填，也不是所有系統的通則。

先看原文 → 對照日期 → 說明為什麼保留未知

格式可能合格，內容卻沒有依據

```
{
  "due_date": "2026-10-08"
}
```

錯誤示例：這是阿明的期限，
不是怡君的期限。

欄位摘錄，不是完整物件。



需要「成品」時，可以請 AI 做 HTML

「把這份研究整理成一個 HTML 報告，加入目錄、並排比較表和可收合的補充說明。」

為什麼現在值得考慮

當主要需求是閱讀、比較與查閱時，HTML 可以提供更合適的介面。

代價

複雜的版面與互動，可能增加生成、檢查與維護成本。自包含的單一 HTML 可直接以瀏覽器開啟；需要公開網址時，再處理託管。

HTML 不是取代 Markdown：是否需要介面 → 成本是否值得 → 仍然要核對內容



換的是查閱方式，不是事實本身

同一份資料，變成方便閱讀的介面

全部工作

阿明 | 整理雨量資料

期限：2026-10-08 | 完成狀態：待確認

怡君 | 製作調查簡報

期限：待確認 | 完成狀態：待確認

來源：分工與期限見 A／B，完成狀態見 C。

切換查看重點
→

只看待確認資訊

怡君的期限尚未決定。

來源：B

阿明、怡君的完成狀態都待確認。

來源：C

教學介面示意；兩個狀態已直接放進 PDF，不需點擊。

畫面可以更好讀，但來源與未知資訊不能消失。

看全部工作 → 看待確認資訊 → 檢查來源沒有少



格式也能拿來交代工作方法

看一個 SKILL.md，就知道 Skill 不只是抽象名詞。

```
---
name: meeting-summary
description: 整理會議紀錄、分工與待確認事項時使用。
---

# 工作方法

讀取指定紀錄，保留資料來源。
整理負責人、工作與期限。
未確認的資訊不得自行補寫。
提醒信只產出草稿，不直接寄出。
```

YAML

名稱與用途

Markdown

工作方法

方法可以重複使用；寫了指南，不等於取得工具權限。
須由支援 Agent Skills 的應用載入。完整技能也可包含其他資源。
先看名稱用途 → 再看做事方法 → 最後確認權限邊界



PDF／DOCX：先確認 AI 實際讀到了什麼

原始文件

預算表 | 單位：千元

項目	金額
設備	80
交通	8

轉換風險示意：漏掉表頭單位

預算表

項目	金額
設備	80
交通	8

數字還在，但單位不見了。

欄位	單位	閱讀順序	來源位置
----	----	------	------

不是只看字有沒有讀出來，而是資訊關係有沒有留下。

虛構轉換風險示意，不是本次 PDF 的實測結果。 | 對照原頁 → 找漏掉的關係 → 再使用轉換結果



先問下一步要做什麼

下一步

寫作、修改與版本控制

交給程式讀固定欄位

視覺呈現與介面查閱

工具設定或技能中繼資料

定稿交付與辦公協作

建議格式

Markdown

JSON

HTML

YAML，遇到再學

PDF／DOCX，依需求選擇

格式可以轉換，真正要守住的是內容、來源與使用目的。 | 不是哪個最專業，而是下一個步驟需要什麼



現在，把整件工作重新走一次



完整示範指令

任務 整理指定的三份會議紀錄，列出負責人、工作、期限與待確認事項。

來源 每一項保留來源；沒有依據的資訊標示未知。

交付 先產出可修改的摘要，再製作一頁報告。

邊界 提醒信只產出文字草稿，不寄出。

異常 文件無法讀取時，明確說明，不自行補寫。

三個檢查點

✓ 沒有編造怡君的期限。

✓ 沒有宣稱兩項工作已完成。

✓ 沒有寄出提醒信。

名詞描述角色；工具連接行動；格式交接成果。

回到任務 → 對照五個交代 → 驗收三件沒有越界的事



四個問題，把這堂課帶回工作現場

這份輸出是給人看，還是給程式讀？
依接手者選筆記、資料或介面。

格式寫對了，內容也對嗎？
怡君的期限仍應是未知。

下一步是修改、展示，還是正式交付？
不必每次都製作三種格式。

轉換後有沒有失去來源、順序或欄位？
A／B／C 與待確認狀態都要留下。

自然語言讓我們表達意圖，格式幫我們固定重要的邊界。
交代清楚之後，還要知道怎麼驗收。

Demo Visual Studio

名詞圖鑑／延伸閱讀

Agentic AI Glossary
geominingical.github.io/Agentic_AI_Glossary

技術釐清來源

OpenAI · Function calling
OpenAI · Structured outputs
Anthropic · Building effective agents
Agent Skills · Specification
Model Context Protocol · Architecture
MDN · Local testing server

名詞描述角色；工具連接行動；
格式交接成果。

